

CITED REFERENCES AND FURTHER READING:

- Swartzrauber, P.N. 1977, "The Methods of Cyclic Reduction, Fourier Analysis and the FACR Algorithm for the Discrete Solution of Poisson's Equation on a Rectangle," *SIAM Review*, vol. 19, pp. 490–501.[1]
- Buzbee, B.L., Golub, G.H., and Nielson, C.W. 1970, "On Direct Methods for Solving Poisson's Equation," *SIAM Journal on Numerical Analysis*, vol. 7, pp. 627–656; see also *op. cit.* vol. 11, pp. 753–763.[2]
- Hockney, R.W. 1965, "A Fast Direct Solution of Poisson's Equation Using Fourier Analysis," *Journal of the Association for Computing Machinery*, vol. 12, pp. 95–113.[3]
- Hockney, R.W. 1970, "The Potential Calculation and Some Applications," *Methods of Computational Physics*, vol. 9 (New York: Academic Press), pp. 135–211.[4]
- Hockney, R.W., and Eastwood, J.W. 1981, *Computer Simulation Using Particles* (New York: McGraw-Hill), Chapter 6.[5]
- Temperton, C. 1980, "On the FACR Algorithm for the Discrete Poisson Equation," *Journal of Computational Physics*, vol. 34, pp. 314–329.[6]

20.5 Relaxation Methods for Boundary Value Problems

As we mentioned in §20.0, relaxation methods involve splitting the sparse matrix that arises from finite differencing and then iterating until a solution is found.

There is another way of thinking about relaxation methods that is somewhat more physical. Suppose we wish to solve the elliptic equation

$$\mathcal{L}u = \rho \quad (20.5.1)$$

where $\mathcal{L}$ represents some elliptic operator and ρ is the source term. Rewrite the equation as a diffusion equation,

$$\frac{\partial u}{\partial t} = \mathcal{L}u - \rho \quad (20.5.2)$$

An initial distribution u *relaxes* to an equilibrium solution as $t \rightarrow \infty$. This equilibrium has all time derivatives vanishing. Therefore, it is the solution of the original elliptic problem (20.5.1). We see that all the machinery of §20.2, on diffusive initial value equations, can be brought to bear on the solution of boundary value problems by relaxation methods.

Let us apply this idea to our model problem (20.0.3). The diffusion equation is

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - \rho \quad (20.5.3)$$

If we use FTCS differencing (cf. equation 20.2.4), we get

$$u_{j,l}^{n+1} = u_{j,l}^n + \frac{\Delta t}{\Delta^2} \left(u_{j+1,l}^n + u_{j-1,l}^n + u_{j,l+1}^n + u_{j,l-1}^n - 4u_{j,l}^n \right) - \rho_{j,l} \Delta t \quad (20.5.4)$$

Recall from (20.2.6) that FTCS differencing is stable in one spatial dimension only if $\Delta t / \Delta^2 \leq \frac{1}{2}$. In two dimensions this becomes $\Delta t / \Delta^2 \leq \frac{1}{4}$. Suppose we try to take the largest possible timestep and set $\Delta t = \Delta^2 / 4$. Then equation (20.5.4) becomes

$$u_{j,l}^{n+1} = \frac{1}{4} \left(u_{j+1,l}^n + u_{j-1,l}^n + u_{j,l+1}^n + u_{j,l-1}^n \right) - \frac{\Delta^2}{4} \rho_{j,l} \quad (20.5.5)$$

Thus the algorithm consists of using the average of u at its four nearest-neighbor points on the grid (plus the contribution from the source). This procedure is then iterated until convergence.

This method is in fact a classical method with origins dating back to the last century, called *Jacobi's method* (not to be confused with the Jacobi method for eigenvalues). The method is not practical because it converges too slowly. However, it is the basis for understanding the modern methods, which are always compared with it.

Another classical method is the *Gauss-Seidel* method, which turns out to be important in multigrid methods (§20.6). Here we make use of updated values of u on the right-hand side of (20.5.5) as soon as they become available. In other words, the averaging is done “in place” instead of being “copied” from an earlier timestep to a later one. If we are proceeding along the rows, incrementing j for fixed l , we have

$$u_{j,l}^{n+1} = \frac{1}{4} \left(u_{j+1,l}^n + u_{j-1,l}^{n+1} + u_{j,l+1}^n + u_{j,l-1}^{n+1} \right) - \frac{\Delta^2}{4} \rho_{j,l} \quad (20.5.6)$$

This method is also slowly converging and only of theoretical interest when used by itself, but some analysis of it will be instructive.

Let us look at the Jacobi and Gauss-Seidel methods in terms of the matrix splitting concept. We change notation and call $\mathbf{u}$ “ $\mathbf{x}$,” to conform to standard matrix notation. To solve

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (20.5.7)$$

we can consider splitting $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U} \quad (20.5.8)$$

where $\mathbf{D}$ is the diagonal part of $\mathbf{A}$, $\mathbf{L}$ is the lower triangle of $\mathbf{A}$ with zeros on the diagonal, and $\mathbf{U}$ is the upper triangle of $\mathbf{A}$ with zeros on the diagonal.

In the Jacobi method we write for the r th step of iteration

$$\mathbf{D} \cdot \mathbf{x}^{(r)} = -(\mathbf{L} + \mathbf{U}) \cdot \mathbf{x}^{(r-1)} + \mathbf{b} \quad (20.5.9)$$

For our model problem (20.5.5), $\mathbf{D}$ is simply the identity matrix. The Jacobi method converges for matrices $\mathbf{A}$ that are “diagonally dominant” in a sense that can be made mathematically precise. For matrices arising from finite differencing, this condition is usually met.

What is the rate of convergence of the Jacobi method? A detailed analysis is beyond our scope, but here is some of the flavor: The matrix $-\mathbf{D}^{-1} \cdot (\mathbf{L} + \mathbf{U})$ is the *iteration matrix* that, apart from an additive term, maps one set of $\mathbf{x}$'s into the next. The iteration matrix has eigenvalues, each one of which reflects the factor by which the amplitude of a particular eigenmode of undesired residual is suppressed during one iteration. Evidently those factors had better all have modulus < 1 for the

relaxation to work at all! The rate of convergence of the method is set by the rate for the slowest-decaying eigenmode, i.e., the factor with the largest modulus. The modulus of this largest factor, therefore lying between 0 and 1, is called the *spectral radius* of the relaxation operator, denoted ρ_s .

The number of iterations r required to reduce the overall error by a factor 10^{-p} is thus estimated by

$$r \approx \frac{p \ln 10}{(-\ln \rho_s)} \quad (20.5.10)$$

In general, the spectral radius ρ_s goes asymptotically to the value 1 as the grid size J is increased, so that more iterations are required. For any given equation, grid geometry, and boundary condition, the spectral radius can, in principle, be computed analytically. For example, for equation (20.5.5) on a $J \times J$ grid with Dirichlet boundary conditions on all four sides, the asymptotic formula for large J turns out to be

$$\rho_s \simeq 1 - \frac{\pi^2}{2J^2} \quad (20.5.11)$$

The number of iterations r required to reduce the error by a factor of 10^{-p} is thus

$$r \simeq \frac{2pJ^2 \ln 10}{\pi^2} \simeq \frac{1}{2}pJ^2 \quad (20.5.12)$$

In other words, the number of iterations is proportional to the number of mesh points, J^2 . Since 100×100 and larger problems are common, it is clear that the Jacobi method is only of theoretical interest.

The Gauss-Seidel method, equation (20.5.6), corresponds to the matrix decomposition

$$(\mathbf{L} + \mathbf{D}) \cdot \mathbf{x}^{(r)} = -\mathbf{U} \cdot \mathbf{x}^{(r-1)} + \mathbf{b} \quad (20.5.13)$$

The fact that $\mathbf{L}$ is on the left-hand side of the equation follows from the updating in place, as you can easily check if you write out (20.5.13) in components. One can show [1-3] that the spectral radius is just the square of the spectral radius of the Jacobi method. For our model problem, therefore,

$$\rho_s \simeq 1 - \frac{\pi^2}{J^2} \quad (20.5.14)$$

$$r \simeq \frac{pJ^2 \ln 10}{\pi^2} \simeq \frac{1}{4}pJ^2 \quad (20.5.15)$$

The factor of two improvement in the number of iterations over the Jacobi method still leaves the method impractical.

20.5.1 Successive Overrelaxation (SOR)

We get a better algorithm — one that was the standard algorithm until the 1970s — if we make an *overcorrection* to the value of $\mathbf{x}^{(r)}$ at the r th stage of Gauss-Seidel iteration, thus anticipating future corrections. Solve (20.5.13) for $\mathbf{x}^{(r)}$, add and subtract $\mathbf{x}^{(r-1)}$ on the right-hand side, and hence write the Gauss-Seidel method as

$$\mathbf{x}^{(r)} = \mathbf{x}^{(r-1)} - (\mathbf{L} + \mathbf{D})^{-1} \cdot [(\mathbf{L} + \mathbf{D} + \mathbf{U}) \cdot \mathbf{x}^{(r-1)} - \mathbf{b}] \quad (20.5.16)$$

The term in square brackets is just the residual vector $\xi^{(r-1)}$, so

$$\mathbf{x}^{(r)} = \mathbf{x}^{(r-1)} - (\mathbf{L} + \mathbf{D})^{-1} \cdot \xi^{(r-1)} \quad (20.5.17)$$

Now *overcorrect*, defining

$$\mathbf{x}^{(r)} = \mathbf{x}^{(r-1)} - \omega(\mathbf{L} + \mathbf{D})^{-1} \cdot \xi^{(r-1)} \quad (20.5.18)$$

Here ω is called the *overrelaxation parameter*, and the method is called *successive overrelaxation* (SOR).

The following theorems can be proved [1-3]:

- The method is convergent only for $0 < \omega < 2$. If $0 < \omega < 1$, we speak of *underrelaxation*.
- Under certain mathematical restrictions generally satisfied by matrices arising from finite differencing, only overrelaxation ($1 < \omega < 2$) can give faster convergence than the Gauss-Seidel method.
- If ρ_{Jacobi} is the spectral radius of the Jacobi iteration (so that the square of it is the spectral radius of the Gauss-Seidel iteration), then the *optimal* choice for ω is given by

$$\omega = \frac{2}{1 + \sqrt{1 - \rho_{\text{Jacobi}}^2}} \quad (20.5.19)$$

- For this optimal choice, the spectral radius for SOR is

$$\rho_{\text{SOR}} = \left(\frac{\rho_{\text{Jacobi}}}{1 + \sqrt{1 - \rho_{\text{Jacobi}}^2}} \right)^2 \quad (20.5.20)$$

As an application of the above results, consider our model problem for which ρ_{Jacobi} is given by equation (20.5.11). Then equations (20.5.19) and (20.5.20) give

$$\omega \simeq \frac{2}{1 + \pi/J} \quad (20.5.21)$$

$$\rho_{\text{SOR}} \simeq 1 - \frac{2\pi}{J} \quad \text{for large } J \quad (20.5.22)$$

Equation (20.5.10) gives for the number of iterations to reduce the initial error by a factor of 10^{-p} ,

$$r \simeq \frac{pJ \ln 10}{2\pi} \simeq \frac{1}{3} pJ \quad (20.5.23)$$

Comparing with equation (20.5.12) or (20.5.15), we see that optimal SOR requires of order J iterations, as opposed to of order J^2 . Since J is typically 100 or larger, this makes a tremendous difference! Equation (20.5.23) leads to the mnemonic that three-figure accuracy ($p = 3$) requires a number of iterations equal to the number of mesh points along a side of the grid. For six-figure accuracy, we require about twice as many iterations.

How do we choose ω for a problem for which the answer is not known analytically? That is just the weak point of SOR! The advantages of SOR obtain only in a fairly narrow window around the correct value of ω . It is better to take ω slightly too large, rather than slightly too small, but best to get it right.

One way to choose ω is to map your problem approximately onto a known problem, replacing the coefficients in the equation by average values. Note, however, that the known problem must have the same grid size and boundary conditions as the actual problem. We give for reference purposes the value of ρ_{Jacobi} for our model problem on a rectangular $J \times L$ grid, allowing for the possibility that $\Delta x \neq \Delta y$:

$$\rho_{\text{Jacobi}} = \frac{\cos \frac{\pi}{J} + \left(\frac{\Delta x}{\Delta y}\right)^2 \cos \frac{\pi}{L}}{1 + \left(\frac{\Delta x}{\Delta y}\right)^2} \quad (20.5.24)$$

Equation (20.5.24) holds for homogeneous Dirichlet or Neumann boundary conditions. For periodic boundary conditions, make the replacement $\pi \rightarrow 2\pi$.

A second way, which is especially useful if you plan to solve many similar elliptic equations each time with slightly different coefficients, is to determine the optimum value ω empirically on the first equation and then use that value for the remaining equations. Various automated schemes for doing this and for “seeking out” the best values of ω are described in the literature.

While the matrix notation introduced earlier is useful for theoretical analyses, for practical implementation of the SOR algorithm we need explicit formulas. Consider a general second-order elliptic equation in x and y , finite differenced on a square as for our model equation. Corresponding to each row of the matrix $\mathbf{A}$ is an equation of the form

$$a_{j,l}u_{j+1,l} + b_{j,l}u_{j-1,l} + c_{j,l}u_{j,l+1} + d_{j,l}u_{j,l-1} + e_{j,l}u_{j,l} = f_{j,l} \quad (20.5.25)$$

For our model equation, we had $a = b = c = d = 1, e = -4$. The quantity f is proportional to the source term. The iterative procedure is defined by solving (20.5.25) for $u_{j,l}$:

$$u_{j,l}^* = \frac{1}{e_{j,l}} (f_{j,l} - a_{j,l}u_{j+1,l} - b_{j,l}u_{j-1,l} - c_{j,l}u_{j,l+1} - d_{j,l}u_{j,l-1}) \quad (20.5.26)$$

Then $u_{j,l}^{\text{new}}$ is a weighted average,

$$u_{j,l}^{\text{new}} = \omega u_{j,l}^* + (1 - \omega)u_{j,l}^{\text{old}} \quad (20.5.27)$$

We calculate it as follows: The residual at any stage is

$$\xi_{j,l} = a_{j,l}u_{j+1,l} + b_{j,l}u_{j-1,l} + c_{j,l}u_{j,l+1} + d_{j,l}u_{j,l-1} + e_{j,l}u_{j,l} - f_{j,l} \quad (20.5.28)$$

and the SOR algorithm (20.5.18) or (20.5.27) is

$$u_{j,l}^{\text{new}} = u_{j,l}^{\text{old}} - \omega \frac{\xi_{j,l}}{e_{j,l}} \quad (20.5.29)$$

This formulation is very easy to program, and the norm of the residual vector $\xi_{j,l}$ can be used as a criterion for terminating the iteration.

Another practical point concerns the order in which mesh points are processed. The obvious strategy is simply to proceed in order down the rows (or columns).

Alternatively, suppose we divide the mesh into “odd” and “even” meshes, like the red and black squares of a checkerboard. Then equation (20.5.26) shows that the odd points depend only on the even mesh values, and vice versa. Accordingly, we can carry out one half-sweep updating the odd points, say, and then another half-sweep updating the even points with the new odd values. For the version of SOR implemented below, we shall adopt odd-even ordering.

The last practical point is that in practice the asymptotic rate of convergence in SOR is not attained until of order J iterations. The error often grows by a factor of 20 before convergence sets in. A trivial modification to SOR resolves this problem. It is based on the observation that, while ω is the optimum *asymptotic* relaxation parameter, it is not necessarily a good initial choice. In SOR with *Chebyshev acceleration*, one uses odd-even ordering and changes ω at each half-sweep according to the following prescription:

$$\begin{aligned}\omega^{(0)} &= 1 \\ \omega^{(1/2)} &= 1/(1 - \rho_{\text{Jacobi}}^2/2) \\ \omega^{(n+1/2)} &= 1/(1 - \rho_{\text{Jacobi}}^2 \omega^{(n)}/4), \quad n = 1/2, 1, \dots, \infty \\ \omega^{(\infty)} &\rightarrow \omega_{\text{optimal}}\end{aligned}\tag{20.5.30}$$

The beauty of Chebyshev acceleration is that the norm of the error always decreases with each iteration. (This is the norm of the actual error in $u_{j,l}$. The norm of the residual $\xi_{j,l}$ need not decrease monotonically.) While the asymptotic rate of convergence is the same as ordinary SOR, there is never any excuse for not using Chebyshev acceleration to reduce the total number of iterations required.

Here we give a routine for SOR with Chebyshev acceleration.

```

sor.h void sor(MatDoub_I &a, MatDoub_I &b, MatDoub_I &c, MatDoub_I &d, MatDoub_I &e,
        MatDoub_I &f, MatDoub_IO &u, const Doub rjac)
Successive overrelaxation solution of equation (20.5.25) with Chebyshev acceleration. a, b, c,
d, e, and f are input as the coefficients of the equation, each dimensioned to the grid size
[0..jmax-1][0..jmax-1]. u is input as the initial guess to the solution, usually zero, and
returns with the final value. rjac is input as the spectral radius of the Jacobi iteration, or an
estimate of it.
{
    const Int MAXITS=1000;
    const Doub EPS=1.0e-13;
    Doub anormf=0.0, omega=1.0;
    Int jmax=a.nrows();
    for (Int j=1; j<jmax-1; j++)
        Compute initial norm of residual and terminate iterations when norm has been reduced by
        a factor EPS.
        for (Int l=1; l<jmax-1; l++)
            anormf += abs(f[j][l]);
    for (Int n=0; n<MAXITS; n++) {
        Doub anorm=0.0;
        Int jsw=1;
        for (Int ipass=0; ipass<2; ipass++) {
            Int lsw=jsw;
            for (Int j=1; j<jmax-1; j++) {
                for (Int l=lsw; l<jmax-1; l+=2) {
                    Doub resid=a[j][l]*u[j+1][l]+b[j][l]*u[j-1][l]
                        +c[j][l]*u[j][l+1]+d[j][l]*u[j][l-1]
                        +e[j][l]*u[j][l]-f[j][l];
                }
            }
            jsw=1-jsw;
        }
    }
}

```

Assumes initial u is zero.

Odd-even ordering.

```

        anorm += abs(resid);
        u[j][1] -= omega*resid/e[j][1];
    }
    lsw=3-lsw;
}
jsw=3-jsw;
omega=(n == 0 && ipass == 0 ? 1.0/(1.0-0.5*rjac*rjac) :
        1.0/(1.0-0.25*rjac*rjac*omega));
}
if (anorm < EPS*anormf) return;
}
throw("MAXITS exceeded");
}

```

The main advantage of SOR is that it is very easy to program. Its main disadvantage is that it is still very inefficient on large problems.

20.5.2 ADI (Alternating-Direction Implicit) Method

The ADI method of §20.3 for diffusion equations can be turned into a relaxation method for elliptic equations [1-4]. In §20.3, we discussed ADI as a method for solving the time-dependent heat-flow equation

$$\frac{\partial u}{\partial t} = \nabla^2 u - \rho \quad (20.5.31)$$

By letting $t \rightarrow \infty$, one also gets an iterative method for solving the elliptic equation

$$\nabla^2 u = \rho \quad (20.5.32)$$

In either case, the operator splitting is of the form

$$\mathcal{L} = \mathcal{L}_x + \mathcal{L}_y \quad (20.5.33)$$

where $\mathcal{L}_x$ represents the differencing in x and $\mathcal{L}_y$ that in y .

For example, in our model problem (20.0.6) with $\Delta x = \Delta y = \Delta$, we have

$$\begin{aligned} \mathcal{L}_x u &= 2u_{j,l} - u_{j+1,l} - u_{j-1,l} \\ \mathcal{L}_y u &= 2u_{j,l} - u_{j,l+1} - u_{j,l-1} \end{aligned} \quad (20.5.34)$$

More complicated operators may be similarly split, but there is some art involved. A bad choice of splitting can lead to an algorithm that fails to converge. Usually one tries to base the splitting on the physical nature of the problem. We know for our model problem that an initial transient diffuses away, and we set up the x and y splitting to mimic diffusion in each dimension.

Having chosen a splitting, we difference the time-dependent equation (20.5.31) implicitly in two half-steps:

$$\begin{aligned} \frac{u^{n+1/2} - u^n}{\Delta t/2} &= -\frac{\mathcal{L}_x u^{n+1/2} + \mathcal{L}_y u^n}{\Delta^2} - \rho \\ \frac{u^{n+1} - u^{n+1/2}}{\Delta t/2} &= -\frac{\mathcal{L}_x u^{n+1/2} + \mathcal{L}_y u^{n+1}}{\Delta^2} - \rho \end{aligned} \quad (20.5.35)$$